```
python3 --version

which python3

mkdir -p ~/AISC

cd ~/AISC

pwd

python -m pip install numpy pandas matplotlib scikit-learn scipy seaborn reportlab

python -c "import numpy, pandas, matplotlib, sklearn, scipy, seaborn, reportlab; print('ALL MAIN LIBRARIES WORKING SUCCESSFULLY')"

nano experiment7.py

python3 experiment7.py



python3 your_code.py
```

Exp 7 – Associative Memory

Autoassociative Memory

```
import numpy as np

# 1. Store patterns
patterns = np.array([
    [1, 1, 1, -1, -1, -1],
    [-1, -1, 1, 1, 1, -1]
])

# 2. Initialize weight matrix
W = np.zeros((6, 6))

# 3. Hebbian learning
for p in patterns:
    W += np.outer(p, p)

# 4. Remove self connections
np.fill_diagonal(W, 0)

print("Weight Matrix:")
print(W)

# 5. Give noisy input
test = np.array([1, -1, 1, -1, -1, -1])

# 6. Recall pattern
state = test.copy()
```

```
for iteration in range(10):
    old_state = state.copy()

    for i in range(6):
        activation = np.dot(W[i], state)

        if activation >= 0:
            state[i] = 1
        else:
            state[i] = -1

    if np.array_equal(state, old_state):
        break

# 7. Display output
print("\nNoisy Input:", test)
print("Recalled Pattern:", state)
print("Original Pattern:", patterns[0])
print("Recall Successful:", np.array_equal(state, patterns[0]))
```

HetroAssociative Memory

```
import numpy as np

# 1. Input patterns
A = np.array([
    [1, 1, -1, -1],
    [-1, -1, 1, 1],
    [1, -1, 1, -1],
    [-1, 1, -1, 1]
])

# 2. Target patterns
B = np.array([
    [1, 1, -1],
    [-1, -1, 1],
    [1, -1, -1],
    [-1, 1, 1]
])

# 3. Initialize weight matrix
W = np.zeros((4, 3))

# 4. Hebbian learning
for i in range(len(A)):
    W += np.outer(A[i], B[i])

print("Weight Matrix:")
```

```
print(W)

# 5. Recall target patterns
print("\nRecall Results")

for i in range(len(A)):
    activation = A[i] @ W
    output = np.where(activation >= 0, 1, -1)

    print("\nInput:", A[i])
    print("Recalled Output:", output)
    print("Target Output:", B[i])
    print("Correct:", np.array_equal(output, B[i]))
```

BAM

```
import numpy as np

# 1. Input patterns (6 neurons)
A = np.array([
    [1, 1, 1, 1, 1, 1],
    [-1, -1, -1, -1, -1, -1],
    [1, 1, -1, -1, 1, 1],
    [-1, -1, 1, 1, -1, -1]
])

# 2. Target patterns (3 neurons)
B = np.array([
    [1, 1, 1],
    [-1, -1, -1],
    [1, -1, 1],
    [-1, 1, -1]
])

# 3. Initialize weight matrix
W = np.zeros((6, 3))

# 4. Hebbian learning
for i in range(len(A)):
    W += np.outer(A[i], B[i])

print("BAM Weight Matrix:")
```

```
print(W)

# 5. Forward recall: A -> B
print("\nForward Recall: A -> B")

for i in range(len(A)):
    output_B = np.sign(A[i] @ W)

    print("\nInput A:", A[i])
    print("Recalled B:", output_B)
    print("Target B:", B[i])
    print("Correct:", np.array_equal(output_B, B[i]))

# 6. Reverse recall: B -> A
print("\nReverse Recall: B -> A")

for i in range(len(B)):
    output_A = np.sign(B[i] @ W.T)

    print("\nInput B:", B[i])
    print("Recalled A:", output_A)
    print("Target A:", A[i])
    print("Correct:", np.array_equal(output_A, A[i]))
```

Hopfield

```
import numpy as np

# 1. Store patterns
patterns = np.array([
    [1, 1, 1, -1, -1, -1],
    [-1, -1, -1, 1, 1, 1]
])

# 2. Noisy test pattern
test = np.array([1, -1, 1, -1, -1, -1])

# 3. Initialize weight matrix
W = np.zeros((6, 6))

# 4. Hebbian learning
for p in patterns:
    W += np.outer(p, p)

# 5. Remove self-connections
np.fill_diagonal(W, 0)

print("Hopfield Weight Matrix:")
print(W)

print("\nSymmetric Matrix:", np.array_equal(W, W.T))
print("Diagonal:", np.diag(W))
```

```
# 6. Define energy function
def energy(state):
    return -0.5 * state @ W @ state

# 7. Start with noisy pattern
state = test.copy()

print("\nInitial State:", state)
print("Initial Energy:", energy(state))

# 8. Iterative neuron updates
for iteration in range(10):

    old_state = state.copy()

    for i in range(6):
        activation = np.dot(W[i], state)

        if activation >= 0:
            state[i] = 1
        else:
            state[i] = -1

    print("Iteration", iteration + 1, ":", state)
    print("Energy:", energy(state))

    # Stop if stable
    if np.array_equal(state, old_state):
        break
```

```
# 9. Display final result
print("\nFinal Recalled Pattern:", state)
print("Expected Pattern:", patterns[0])

print("Recall Successful:",
      np.array_equal(state, patterns[0]))
```

Exp-6 Cluster:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score
from scipy.cluster.hierarchy import linkage, dendrogram

# --------------------------------
# 1. Sample Dataset
# --------------------------------

X = np.array([
    [1, 2],
    [2, 3],
    [3, 2],
    [8, 8],
    [9, 8],
    [8, 9],
    [15, 15],
    [16, 15],
    [15, 16]
])

# --------------------------------
# 2. K-Means Clustering
# --------------------------------

km = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
k_labels = km.fit_predict(X)

print("========== K-MEANS CLUSTERING ==========")

print("Cluster Labels:")
print(k_labels)

print("\nCluster Centers:")
print(km.cluster_centers_)

# --------------------------------
# 3. Hierarchical Clustering
# --------------------------------

hc = AgglomerativeClustering(
    n_clusters=3,
    linkage='ward'
)

h_labels = hc.fit_predict(X)

print("\n========== HIERARCHICAL CLUSTERING ==========")

print("Cluster Labels:")
print(h_labels)

# --------------------------------
# 4. Dendrogram Visualization
# --------------------------------
```

```
Z = linkage(X, method='ward')

plt.figure(figsize=(8, 5))
dendrogram(Z)

plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# --------------------------------
# 5. Evaluation Metrics
# --------------------------------

print("\n========== EVALUATION METRICS ==========")

# K-Means Evaluation
print("\nK-Means Results")

print("Silhouette Score:",
      silhouette_score(X, k_labels))

print("Davies-Bouldin Score:",
      davies_bouldin_score(X, k_labels))

print("Inertia:", km.inertia_)

# Hierarchical Evaluation
print("\nHierarchical Results")
```

```
print("Silhouette Score:",
      silhouette_score(X, h_labels))

print("Davies-Bouldin Score:",
      davies_bouldin_score(X, h_labels))
```

Exp-3 Adder

```
import numpy as np
from sklearn.neural_network import MLPClassifier

# Input combinations
X = np.array([[0,0], [0,1], [1,0], [1,1]])

# Half Adder outputs: SUM and CARRY
y = np.array([
    [0,0],
    [1,0],
    [1,0],
    [0,1]
])

# Create and train ANN
model = MLPClassifier(hidden_layer_sizes=(8,),
                      max_iter=5000,
                      random_state=42)

model.fit(X, y)

# Predict output
pred = model.predict(X)

print("Half Adder Output:")
print(pred)
```

```
import numpy as np
from sklearn.neural_network import MLPClassifier

# Input combinations
X = np.array([
    [0,0,0], [0,0,1],
    [0,1,0], [0,1,1],
    [1,0,0], [1,0,1],
    [1,1,0], [1,1,1]
])

# Full Adder outputs: SUM and CARRY
y = np.array([
    [0,0], [1,0],
    [1,0], [0,1],
    [1,0], [0,1],
    [0,1], [1,1]
])

# Create and train ANN
model = MLPClassifier(hidden_layer_sizes=(8,),
                      max_iter=5000,
                      random_state=42)

model.fit(X, y)

# Predict output
pred = model.predict(X)

print("Full Adder Output:")
print(pred)
```

Exp – 4 Single Layer Preceptron

```
import numpy as np

X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 0, 0, 1])

w = np.zeros(2)
b = 0
lr = 0.1

for epoch in range(10):
    for i in range(4):
        net = np.dot(X[i], w) + b

        pred = 1 if net >= 0.5 else 0

        error = y[i] - pred

        w = w + lr * error * X[i]
        b = b + lr * error

print("Predicted Output:")
for i in range(4):
    net = np.dot(X[i], w) + b
    pred = 1 if net >= 0.5 else 0
    print(pred)
```

Exp - 5 Decision Tree Classification

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Dataset
X = np.array([[1,2], [2,3], [3,4], [6,7], [7,8], [8,9]])
y = np.array([0, 0, 0, 1, 1, 1])

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create and train model
model = DecisionTreeClassifier(criterion="entropy", random_state=42)
model.fit(X_train, y_train)

# Prediction
pred = model.predict(X_test)

print("Actual Output:", y_test)
print("Predicted Output:", pred)
print("Accuracy:", accuracy_score(y_test, pred))

# Visualize Decision Tree
plt.figure(figsize=(8, 5))
plot_tree(model, filled=True)
plt.show()
```

Exp 8 – Fuzzy

```
import numpy as np
import matplotlib.pyplot as plt

# Triangular Membership Function
def triangular_mf(x, a, b, c):
    if x <= a or x >= c:
        return 0
    elif x == b:
        return 1
    elif x < b:
        return (x - a) / (b - a)
    else:
        return (c - x) / (c - b)

# Input value
x = 30

# Fuzzy sets
low = triangular_mf(x, 0, 0, 50)
medium = triangular_mf(x, 0, 50, 100)
high = triangular_mf(x, 50, 100, 100)

# Display membership values
print("Low Membership:", low)
print("Medium Membership:", medium)
print("High Membership:", high)

# Plot membership functions
values = np.arange(0, 101)
```

```
low_set = [triangular_mf(i, 0, 0, 50) for i in values]
medium_set = [triangular_mf(i, 0, 50, 100) for i in values]
high_set = [triangular_mf(i, 50, 100, 100) for i in values]

plt.plot(values, low_set, label="Low")
plt.plot(values, medium_set, label="Medium")
plt.plot(values, high_set, label="High")

plt.xlabel("Input")
plt.ylabel("Membership")
plt.title("Fuzzy Membership Functions")
plt.legend()
plt.grid()
plt.show()
```